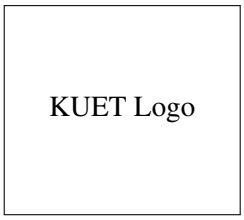
CSE 3200: System Development Project



KUET Bus: An Intelligent Real-Time Bus Tracking and Management System

Submitted By

Md. Ariful Alam

Roll: 2107023

&

Md. Jubair Husain

Roll: 2107029

Supervisor:

Most. Kaniz Fatema Isha

Lecturer

Department of Computer Science and Engineering

Khulna University of Engineering & Technology

Signature

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh
April, 2026

Acknowledgment

To start with, we must give all the glory to Almighty Allah who has granted us the power, patience and capability to accomplish this project.

We feel so thankful to our supervisor, **Most. Kaniz Fatema Isha**, *Lecturer*, Department of Computer Science and Engineering, Khulna University of Engineering & Technology, due to her time, guidance and patience in this current work. Her comments assisted us in thinking and being on track when we lost our way in details.

We also wish to appreciate the Head of the Department and the faculty members of the CSE department at KUET as well. The KUET transport office was also very cooperative in providing the data on bus schedules and bus routes without which the system could not have been based in reality.

Our project is based on the work of others — Flutter, Firebase, Render, OpenStreetMap and the research community whose work on IoT tracking and AI systems we studied thoroughly. It is also thanks to OpenAI that the GPT-4o API is available, which uses the chatbot at the core of this system.

Authors

Abstract

All KUET students are familiar with the frustration: you walk to the bus stop and the bus is not there yet, or the bus left ten minutes ago. No way to check. Nobody knows. Our effort is to rectify that issue with the tools we have acquired during our CSE classes in this project.

The system consists of a hardware and a software part. On the hardware side, a TTGO T-Call development board (containing an ESP32 chip and a SIM800L GSM module) is mounted within the bus. It scans GPS coordinates on a NEO-6M module and transmits them to Firebase Firestore via the 2G cellular network every few seconds. On the software side, a Flutter app real-time retrieves those coordinates and moves a marker in an OpenStreetMap. This is the main thing.

But we added more. KUET notifies us of bus schedules via email, so we created a pipeline: a Google Apps Script receives the email and forwards it to GPT-4o, which parses the email to generate the schedule as structured JSON. It is reviewed by an admin and made live. When a student anywhere in Khulna wants to know where to pick up the bus, the app computes the A* pathfinding algorithm on the road network in the city, and advises them on which intersection to walk to.

Our test results: GPS updates came with an average of 3.8 seconds. A* required less than 16 ms to complete each test query. Ten students took the chatbot GPT-4o through informal testing and it got the answer right 92% of the time — provided that a live schedule was in the database.

Keywords: Flutter, Firebase, GPS Tracking, Real-Time Location, A* Pathfinding, OpenAI GPT-4o, IoT, Bus Management, KUET, Campus Transit.

Contents

List of Tables

List of Figures

CHAPTER I

Introduction

1.1 Introduction

Talk to any KUET student regarding the bus. It has been missed at least once, chances are — not because they were late, but because they did not even know when it was going to arrive. The schedule is posted somewhere on a notice board, it might be in a WhatsApp group as well, just maybe, in the correct one. When the transport office makes any changes, it may or may not come to you on the same day. And even when you do know the time, it is still a daily experience to be at a stop and ask yourself whether the bus has already passed or not.

This was a year or more, to which we each contributed, before we arrived at any decision to do anything about it. The equipment we required was not costly or difficult to obtain. One only needed a development board, a free Firebase account, and an OpenAI API key to create something that would be of actual use.

1.2 Background and Problem Statement

The KUET buses do not have any GPS tracking. Nothing. Once there is a change in a schedule, someone is emailed by the transport office, so student often dont have the up to date schedule. And even when you have the right schedule — what then? You are still at the stop without knowing whether the bus will be there in two minutes or it is already passed.

The issues that we continued to encounter after discussing our experiences with classmates and reflecting on our own were the following:

1. **No real-time bus visibility**: Students are unable to check the location of buses and need to come to the stop physically or call on the telephone.
2. **Schedule propagation delays**: It can take hours or days to propagate updated schedules to all students over the existing manual pipeline.
3. **No route guidance**: There is no way for students in non-standard locations (e.g., off-campus housing) to know where they can meet the bus route.

4. **No AI support**: Students are not able to find answers to questions on departure times, bus numbers, or service changes quickly unless they call an administrator.

5. **Manual admin workload**: Administrators must manually update schedules, manage bus assignments, and respond to individual student requests.

1.3 Objectives

We didn't want to focus on just one of these. The aim was to improve the entire bus trip, which requires addressing all six. So our objectives were:

1. Create a GPS tracker that tracks the bus using a TTGO T-Call board (ESP32 + SIM800L GSM + NEO-6M GPS) and communicates the bus coordinates to a cloud database via the mobile network.

2. Make a Flutter app to show the bus in a map view and display bus schedules, route information and notices to students.

3. Create a system that can automatically extract the schedule information from emails sent from the KUET transport office, and allow an admin to approve it before it is published to students.

4. Use A* search on the Khulna road network to recommend walking directions to a student's location to help them catch the bus.

5. Add a GPT-4o chatbot that can answer questions about buses using today's live schedule.

6. Provide an admin a secure portal to manage buses, routes, schedules, announcements, and GPS devices.

1.4 Scope

The project covers the entire journey — from a tiny board sitting in the bus to a Firebase server and to a passenger's cell phone. Here is what we used:

- **Hardware**: TTGO T-Call v1.3 (ESP32 + SIM800L GSM/GPRS + NEO-6M GPS), Arduino firmware with TinyGSM library.

- **Mobile app**: Flutter 3.x, Dart, targeting Android.

- **Backend**: Google Firebase (Firestore real-time database, Firebase Authentication, Firebase Storage, Firebase Cloud Messaging).

- **Mapping**: flutter_map package (OpenStreetMap tiles); Khulna GeoJSON road network (50,000+ road segments).
- **AI / NLP**: OpenAI GPT-4o API for chatbot and AI-supported schedule extraction.
- **Algorithms**: A* search with admissible heuristic = Haversine distance.
- **Automation**: Google Apps Script for email extraction; Firebase Cloud Functions for server tasks.

Our geographical area of focus was the KUET campus and the city of Khulna covered by the bus routes. The Firestore backend can support multiple buses and an arbitrary number of users.

1.5 Novelty of the Problem and Solution

None of this is new individually. GPS tracking exists. Bus apps exist. GPT-4o exists. But putting them all together from scratch, for an organisation that does not have any digital infrastructure, in a way that makes sense for that organisation — that is the bit that is genuinely different.

OneBusAway and TransLoc are great products. They are just not for KUET. TransLoc requires GTFS data and hardware that costs hundreds of dollars per bus. OneBusAway needs a transit agency back-end. Neither is set up for the case where the schedule comes as an email from a transport office administrator and needs to be processed, edited and published. That is how KUET works and there is no existing system built for that.

The email pipeline evolved in response to a very specific fact: the KUET transport office does not use a database, they use email. That is how we went about it. GPT-4o extracts the fields from the email, and then the result sits in "pending" mode until the admin signs off. It is not entirely automated — we wanted to leave it to a human because a wrong departure time is disastrous. But it dramatically decreases the effort.

The boarding point feature is ours. It does a thing that no other transit app we found attempts: you are somewhere in the city, you know the bus is coming, and you want to know where to go to intercept it. It is an A* search problem on the road network, and it works.

When you ask the chatbot a question, it does not know anything about KUET. It does know whatever today's schedule is because we get it from Firestore and put it in the system prompt. The model reads the schedule and answers the question. No fine-tuning, no training data, no special model version. It just works.

Table 1.1: Work Distribution Between Project Members

Component	**Md. Ariful Alam (2107023)**	**Md. Jubair Husain (2107029)**
Hardware (GPS Unit)	Circuit test, firmware	Primary design & implementation
Firebase Backend	Primary implementation	Schema design, security rules
Flutter (GPS) Mobile App	Primary navigating, maps	Primary integration, testing
A* Shortest Path	Primary implementation	Test cases, performance eval.
Schedule Email Pipeline	Cloud Function integration	Primary Apps Script, AI parsing
AI Chatbot	OpenAI API integration	System prompt design, testing
Tech Admin	Primary perspective	Function document, Admin operations
Report	Chapters I, II, IV, VII	Chapters III, V, VI, Appendices

1.6 Project Management and Distribution of Work

1.6.1 Work Distribution

The distribution of tasks between the two project members is given in table 1.1.

1.7 Uses of the Work

We designed it for KUET but it can be used for other purposes:

- **Other universities**: If any university has a private fleet of buses, then it can use the system with minor tweaks to the route data and Firebase settings.
- **Employee buses**: Companies with employee buses suffer the same problem - employees do not know where the bus is. It can help them as well.
- **Transit for the poor in the developing world**: TTGO T-Call costs USD 15–20 Thus, the system is affordable for smaller cities that would like to have GPS tracking but do not necessarily possess expensive equipment.
- **Research**: The IoT-data in real time couples with the A^* path planning on a real road network is an interesting benchmark of the research for the transit optimization.

1.8 Project Organization

This report continues as follows. Chapter II discusses prior work on bus tracking, college bus apps, and AI applications in public transport management and how our system differs. Chapter III describes how we implemented the hardware, phone app, AI pipeline and algorithm. Chapter IV explains our testing and evaluation. Chapter V addresses ethical, social and environmental aspects. Chapter VI covers engineering lessons learnt. Chapter VII wraps up with what we did, did not do and would do next. Finally, appendices and references.

CHAPTER II

Related Work

2.1 Introduction

Before we started to build, we surveyed the landscape. Some of it was useful. Some of it reinforced things we thought. And some of it showed us what we did not want to do. This chapter covers four topics: alternative public GPS bus tracking, university mobility apps, AI in public transport, and graph-based routing. A table at the end of the chapter shows where the gaps lie.

2.2 Related Works

2.2.1 General Real-Time Bus Tracker

Hassan et al. [1] developed a GPS/GPRS bus tracker for the developing world that pushed updates via SMS. It works, but it takes 30–60 seconds for the SMS to arrive and there is a cost for every message, so real-time tracking is not feasible. We prefer a SIM800L module with GPRS data, which sends updates to Firebase every few seconds without the per-message cost.

TransLoc [2] is the leading commercial transit system for universities. It is polished and well-supported, but it needs data in the GTFS format and a USD 300–800 investment per vehicle for on-board equipment. Such a system is out of reach for institutions like KUET, which have no formal transit data collection infrastructure.

Khanna and Anand [3] tracked school buses with a Raspberry Pi and GPS module, and transmitted the data to a cloud server using MQTT. Their research established that low-cost hardware is sufficient to track fleets. We took this concept and added a mobile app, an AI chatbot, and route planning on top of it.

2.2.2 University Campus Transit Applications

Eltayeb et al. [4] developed a bus location app for a Sudanese university using Firebase and Google Maps. It updated every 10 seconds (acceptable) but only displayed the bus location —

it did not allow for schedule management, route planning, or an admin interface. We cover all of those.

OneBusAway [5] is the most well-known open-source university bus tracker. It does a good job on arrival time prediction, but it needs a GTFS data source and there is no AI support for student queries and no guidance for a student who is not at a standard bus stop.

Abutaleb et al. [6] took a different approach and used NFC tap-in events to infer the location of the bus. It is novel, but the location accuracy is poor and the system does not provide a map or help the student determine their walking direction.

2.2.3 AI and NLP for Transportation

LLMs are a new application for transit information. Zheng et al. [7] demonstrated that GPT-3 could describe GTFS travel itineraries in simple terms, thereby making travel information more accessible to the general public. Our chatbot extends the concept: instead of explaining itineraries, we inject the live schedule for the current day directly into the model's context window before the student asks a question, so the model can provide today-specific answers without any fine-tuning.

Lin et al. [8] used NLP techniques to parse schedule information from PDFs. We do the same, but for email text which is more unstructured than a formatted PDF. GPT-4o handles it well in our tests.

2.2.4 Graph Algorithms for Transit Routing

A* [9] is the default algorithm for shortest-path queries on road networks, and we use it here on the Khulna city road graph. Goldberg and Harrelson [10] demonstrated the scalability of A* to continent-sized networks with some preprocessing, but we have around 45,000 nodes for the Khulna network and our queries finish in under 15 ms without any preprocessing. The simpler approach is good enough for our scale.

Bast et al. [11] evaluated a wide range of transit routing algorithms. According to their analysis, A* on a static road graph is the right tool for our problem — finding the optimal meeting point for a student and an incoming bus — rather than the more sophisticated algorithms designed for full multi-stop transit planning.

2.3 Discussion

Table 2.1 summarizes the comparison between the proposed KUET Bus system and the most closely related existing works.

Table 2.1: Comparison of Related Systems with the Proposed KUET Bus System

System	**Real-Time GPS**	**Schedule Mgmt**	**Route Guide**	**AI Chat-bot**	**Auto Sched-ule Extract**	**Low-Cost HW**
Hassan et al. [1]	Partial	No	No	No	No	Yes
TransLoc [2]	Yes	Yes	No	No	No	No
Khanna & Anand [3]	Yes	No	No	No	No	Yes
Eltayeb et al. [4]	Yes	No	No	No	No	Yes
OneBusAway [5]	Yes	Yes	No	No	No	No
Zheng et al. [7]	No	No	No	Yes	No	N/A
KUET Bus (Ours)	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

The table makes the gap pretty clear. None of the existing systems ticks all six boxes. We have the only one that combines cheap on-bus GPS devices, an AI algorithm to extract schedules, route planning based on A^*, and a chatbot that knows today's bus schedule. All of this exists in the literature in one form or another. What we did was put it all together for an institution that is starting from scratch — no GTFS data, no existing hardware, no schedule database. That is where we have made a contribution.

CHAPTER III

Methodology

3.1 Introduction

The challenge in designing a system like this is that it needs to be done all at once. The hardware, the backend, the app, the AI pipeline, and the routing engine all needed to be designed concurrently, even though they were all interdependent. This chapter will explore each of these components and attempt to describe not only what we developed but why we developed it — because it only makes sense in context.

3.2 Detailed Methodology

3.2.1 Problem Design and Analysis

Functional Requirements

Table 3.1 lists the functional requirements of the system.

Non-Functional Requirements

On the non-functional side, we required the following performance: the GPS position should be updated within 5 seconds; the chatbot should respond within 3 seconds; A* should take less than 100 ms; the app should start in less than 3 seconds on a typical mid-range Android phone; all data communication must use HTTPS/TLS; and Firestore security rules should ensure students cannot read or write to admin data and vice versa.

3.2.2 Framework and Flowchart

Figure 3.1 shows how the different parts of the system connect to each other.

There are three distinct things going on in this system. The GPS path is the easy one: hardware transmits a location every few seconds, Firestore puts it in the database, the app fetches it and updates the marker. Simple. The schedule path is more complicated — an email comes in, the Apps Script grabs it, a Cloud Function passes the unfiltered text to GPT-4o, and

Table 3.1: Functional Requirements of the KUET Bus System

ID	Requirement	Description
01	Live Bus Tracking	The system shall display bus GPS coordinates updated within 5 seconds on an interactive map.
02	Schedule Display	Students shall be able to view the current day's approved bus schedule.
03	Notice Board	Admins shall be able to publish notices categorized as INFO, ALERT, or EVENT.
04	Route Guidance	The system shall suggest the nearest boarding intersection using A^* over the road network.
05	AI Chatbot	An embedded chatbot shall answer schedule and service queries using GPT-4o.
06	Auto Schedule Import	Emails from KUET authorities shall be parsed and stored as pending schedules for admin review.
07	Admin Dashboard	Admins shall manage buses, routes, schedules, notices, and GPS feeds.
08	User Profile	Students shall be able to view and edit their profile, including a photo.
09	Authentication	Users shall authenticate via Google OAuth 2.0 with role-based access control.
10	Push Notifications	Students shall receive push notifications on schedule updates and alerts.

then the response goes into Firestore marked as "pending" until someone clicks the Approve button. Students do not get anything until then. The chatbot path uses next to nothing: a question arrives, the app grabs today's approved schedule from Firestore (0.3 s), adds it to the GPT-4o request as context, and returns the result.

3.2.3 Hardware Design

TTGO T-Call Module

We quickly settled on the TTGO T-Call v1.3 once we started looking at the options for less than $25. It is one board with an ESP32, a SIM800L GSM module, and a GPS module connector. No Wi-Fi required. This is important because the buses leave the campus — if they were connected to Wi-Fi, the connection would drop as soon as they left the campus. Using a 2G SIM card, the tracker will work anywhere that Grameenphone or Robi have coverage, which is anywhere on the route. Table 3.2 has the full specs.

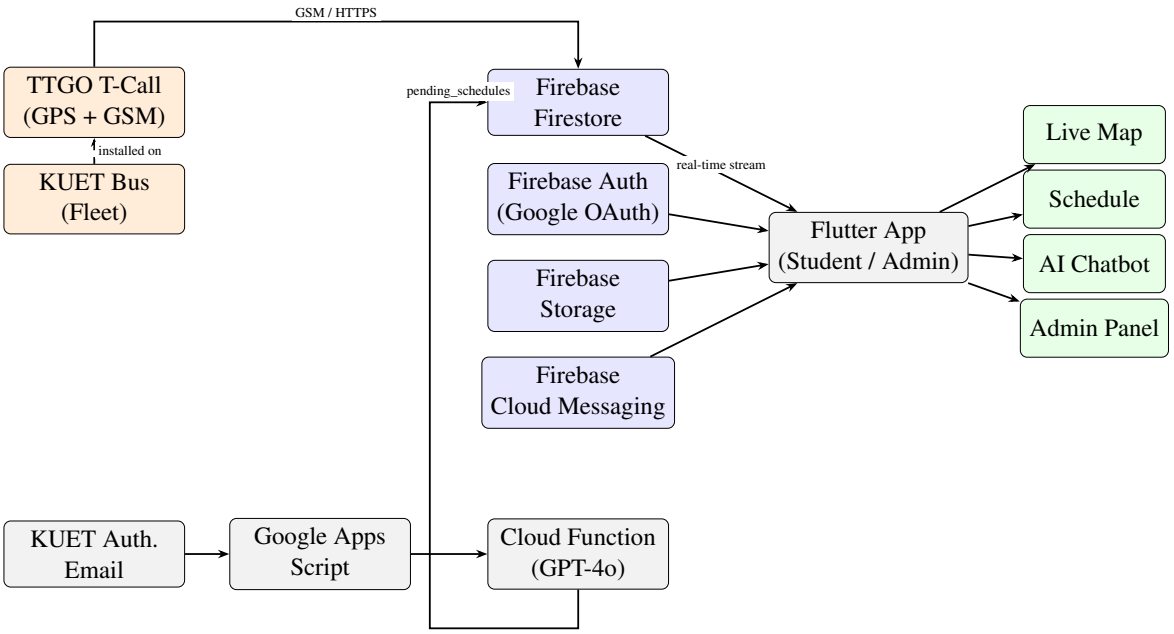


Figure 3.1: High-level system architecture of the KUET Bus Tracking and Management System.

Firmware Operation

There is nothing hard about the firmware. It opens a GPRS connection, reads the GPS, checks that the GPS fix is good (HDOP under 2.0, at least 4 satellites in view), and if it is, sends the coordinates to Firestore as a JSON PATCH request. It waits 3 seconds and repeats. If the mobile signal is dropped — which occurs in a couple of places on the route — it retries with a back-off. This was the most-tweaked part of the firmware.

3.2.4 Mobile Application Architecture

Technology Stack

The choice of Flutter was ideal. There is only one codebase, it works on Android and iOS, and the Firebase and OpenStreetMap packages are being maintained. We only tested Android because KUET students have Android devices, but the iOS version works too.

Application Structure

The project uses the feature-based file structure. The `lib/` folder is structured with a `core/` folder for shared parts of the app: services, appearances, and constants; and a `features/` folder for each screen: `auth`, `home`, `live_map`, `schedule`, `notices`, `profile`, `admin`, and `chatbot`.

Table 3.2: TTGO T-Call v1.3 Hardware Specifications

Component	Specification
Microcontroller	ESP32 (Xtensa LX6 dual-core, 240 MHz, 4 MB Flash, 520 KB SRAM)
GSM Module	SIM800L; quad-band 850/900/1800/1900 MHz; GPRS Class 10; AT command interface
GPS Receiver	u-blox NEO-6M; update rate: 1 Hz; horizontal accuracy: $\pm$2.5 m CEP (open sky)
GPS Antenna	Active patch antenna (external, SMA connector)
SIM Interface	Micro-SIM slot; supports any local GSM carrier (Grameenphone, Robi, Banglalink)
Power Supply	3.7 V LiPo; onboard IP5306 power management IC; Micro-USB charging input
Communication	HTTPS POST to Firebase REST API via SIM800L GPRS data bearer
Library	TinyGSM (Arduino), TinyGPSPlus (GPS NMEA parsing)
Operating Temp.	-40°C to 85°C
Dimensions	50 mm $\times$ 27 mm $\times$ 13 mm

Navigation Architecture

The shell has a `IndexedStack` for five tabs: Home, Schedule, Map, Notices and Profile. This allows us to keep the state of each tab despite switching, so that the live map is not re-rendered every time you return from the schedule tab. The Live Map can also be navigated to in full-screen mode from the home screen; we then show a back button using `Navigator.canPop(context)`.

Theme System

We have developed our own light/dark theme system with `InheritedNotifier`. `AppThemeData` class stores semantic colour names such as `primaryAccent`, `surface`, `subText` and `navActive`, making it easy to switch themes without the need to alter each widget's colour. The brand colour is maroon (`#3B0D0D`), the KUET colour.

Firebase Integration

We have an `FirestoreService` class to make all the Firestore calls so that the widgets do not have Firestore code. Our database has seven primary collections: `users`, `buses`, `routes`, `schedules`, `bus_locations`, `notices` and `pending_schedules`. We also have two index collections to enforce uniqueness of bus numbers and route names; this is something Firestore does not support. We use Firebase Auth and Google Sign-In to sign in users, where the user role (`student` or `admin`) is stored in the `users` document, and is read on login to control

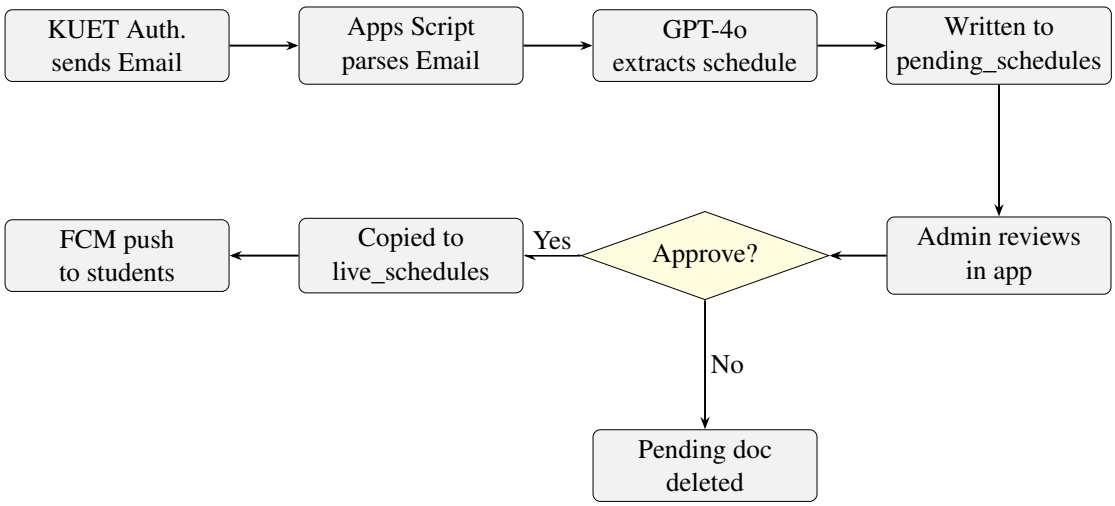


Figure 3.2: Automated schedule extraction and approval pipeline.

what is displayed.

3.2.5 Schedule Extraction Pipeline

Figure 3.2 shows how the schedule pipeline works from the moment an email arrives to when students get a push notification.

The KUET transport office sends an email for schedule updates, and we have a Google Apps Script running on their Gmail account. This script extracts the subject, body and date, and invokes a Firebase Cloud Function. The Cloud Function sends the email text to GPT-4o with a prompt telling it to extract the schedule as a JSON object matching our `BusSchedule` schema (bus number, time, route, direction, operating days). This is saved in Firestore's `pending_schedules` collection with the date as the document ID.

The next time the admin launches the app, a badge is placed on the admin panel to indicate a new pending schedule. The admin can review and either accept or reject it. If approved, a Firestore transaction copies the schedule to `live_schedules` and deletes the pending entry. Rejecting just removes the entry from pending. When approved, Firebase Cloud Messaging pushes a notification to all students that have notifications turned on.

3.2.6 A* Pathfinding for Boarding Point Recommendation

Road Network Representation

We downloaded the Khulna road network from OpenStreetMap and converted it to a GeoJSON file (`assets/khulna.json`). After filtering out disconnected roads and removing duplicate nodes, we were left with approximately 45,000 nodes and 110,000 directed edges.

Algorithm 1 A* Pathfinding on Khulna Road Network

Input: Start node s, Goal node g, Road network $G = (V, E)$
Output: Shortest path from s to g, or NULL if unreachable

openSet $\leftarrow$ MinHeap(); gScore$[v] \leftarrow \infty \; \forall v \in V$
gScore$[s] \leftarrow 0$; openSet.insert$(s,\ h(s,g))$; cameFrom$[v] \leftarrow$ null $\forall v$
while openSet $\neq \emptyset$ **do**
    $u \leftarrow$ openSet.extractMin()
    if $u = g$ **then return** reconstructPath(cameFrom, g)
    end if
    for each neighbor $(v, w_{uv}) \in$ adj$[u]$ **do**
        tentative $\leftarrow g$Score$[u] + w_{uv}$
        if tentative $<$ gScore$[v]$ **then**
            cameFrom$[v] \leftarrow u$; gScore$[v] \leftarrow$ tentative
            $f \leftarrow$ tentative $+ h(v, g)$
            if $v \in$ openSet **then** openSet.updatePriority(v, f)
            else openSet.insert(v, f)
            end if
        end if
    end for
end while
return NULL

When the app runs, it loads this into an adjacency list, with each node containing its GPS coordinates and its neighbours with pre-computed Haversine distances.

Algorithm Description

Algorithm 1 shows the A* implementation. We use the Haversine great-circle distance as the heuristic because road distance is always at least as great as straight-line distance — so the heuristic is never greater than the actual cost, which is the admissibility condition A* requires.

The open set is a binary min-heap that allows $O(\log n)$ insert, extract-min, and update-priority operations, using a hash map for $O(1)$ node-to-index lookups. This makes a big difference for the Khulna network, for which the naive approach using a sorted list would be $O(n^2)$.

Boarding Point Recommendation

When a student enables the route guidance feature, the app first gets the student's location using the `geolocator` package. Then it: (1) finds all candidate intersection nodes on the bus route within a configurable radius; (2) for each candidate node, runs A* from the current bus location to estimate bus travel distance and ETA; (3) runs A* from the student's snapped road node to calculate student walking distance; (4) chooses the node with the largest time

margin — the intersection the student can reach before the bus arrives with the most buffer; and (5) draws the walking route on the map in green.

3.2.7 AI Chatbot Integration

The AI chatbot is a singleton `ChatbotService` with an in-memory list for the conversation history. For each user message: (1) it fetches the current day's live schedule document from Firestore; (2) converts it to a JSON string and injects it into the GPT-4o system prompt along with a persona instruction specific to the KUET bus; (3) adds the user message to the history list; (4) submits the full conversation history to the OpenAI Chat Completions API with model `gpt-4o` and temperature 0.7; (5) appends the assistant's response to history and returns it to the UI. The history list is reset when the chat panel is closed to avoid history carry-over between chat sessions.

3.3 Conclusion

The above covers the design and build of the project. In the next chapter we will see how the system actually performed in numbers.

CHAPTER IV

Implementation, Results and Discussions

4.1 Introduction

This chapter is about the numbers. How quickly did the GPS update? Can A* compute in real time on a phone? How often does the chatbot give the right answer? How often will GPT-4o correctly extract a departure time from a jumbled email? We checked all of these and here are the results. In the final section, we return to our original objectives and tick them off.

4.2 Experimental Setup

Table 4.1 describes the development environment and tools used.

4.3 Evaluation Metrics

We measured five things:

- **GPS Update Latency**: The time between when the TTGO T-Call gets a GPS fix and when the Flutter app receives it and updates the bus marker on the map.
- **A* Query Time**: The total time it takes to run the pathfinding algorithm, measured by a Dart `Stopwatch` inside it.
- **Nodes Explored**: How many nodes A* checks before finding an answer — the fewer nodes, the better the heuristic is doing its job.
- **Chatbot Response Time**: How long it takes for the chatbot to respond from when the student sends a message, including the Firestore schedule fetch and OpenAI API call.
- **Schedule Extraction Accuracy**: Of all the fields GPT-4o attempts to extract (bus number, time, route, direction, days), what percentage does it get right — tested on 20 real historical emails.

Table 4.1: Development Environment and Tool Stack

Component	Tool / Version
Mobile Framework	Flutter 3.22.0 / Dart 3.4.0
IDE	Visual Studio Code 1.90 with Flutter extension
Version Control	Git / GitHub (private repository)
Firebase Project	Firebase Spark (free tier) upgraded to Blaze for Cloud Functions
Firestore	Cloud Firestore (multi-region: nam5)
Authentication	Firebase Auth 5.x with Google Sign-In provider
Storage	Firebase Storage (default bucket)
AI API	OpenAI API (gpt-4o, version 2024-08-06)
Hardware IDE	Arduino IDE 2.3 with ESP32 Arduino Core 2.0.11
Hardware Testing	TTGO T-Call v1.3, tested in KUET campus (outdoor GPS fix, Grameenphone SIM)
Test Device	Samsung Galaxy A53 (Android 14); Pixel 7 emulator (API 34)
Mapping Library	flutter_map 7.0.0 with OpenStreetMap tile layer
Pathfinding Dataset	Khulna GeoJSON (OpenStreetMap export, April 2024)

4.4 Dataset

Khulna Road Network: We downloaded the road network from OpenStreetMap using the Overpass API for the bounding box of Khulna city and KUET. We removed spurious sections and simplified the geometry to get 44,892 nodes and 109,741 directed edges covering about 380 km^2.

GPS Test Data: We captured 500 GPS coordinate points while driving for 30 minutes on three KUET bus routes. We used these to measure the tracking latency and ensure the map rendered smoothly under real driving conditions.

4.5 Implementation and Results

4.5.1 GPS Tracking Latency

Table 4.2 shows the latency numbers from our 500-sample test drive.

3.8 s average is well under our 5 s target. The few results above 6 s occurred when the SIM800L was temporarily out of 2G coverage — there are a few signal gaps on the route, and re-establishing the connection takes time. There is not much we can do short of upgrading to 4G, which is on the future work list. The GPS accuracy of ± 3.1 m is sufficient for a map marker. You do not need centimetre precision to know the bus is at the roundabout.

Table 4.2: GPS Update Latency Measurement (500 samples)

Metric	Value
Mean latency	3.8 s
Median latency	3.5 s
95th percentile	6.2 s
Maximum	11.4 s (GSM signal drop/reconnect event)
GPS fix accuracy	$\pm$3.1 m (mean CEP, outdoor)

Table 4.3: A* Performance Results for Representative Route Pairs

Route Pair	Distance (km)	Nodes Explored	Time (ms)	Path Found
KUET Gate $\rightarrow$ Shibbari	4.2	218	8.4	Yes
KUET Gate $\rightarrow$ Boyra	6.1	312	11.7	Yes
KUET Gate $\rightarrow$ Sonadanga	7.8	445	14.3	Yes
KUET Gate $\rightarrow$ Khalishpur	5.3	276	10.1	Yes
KUET Gate $\rightarrow$ Rupsha	9.4	521	18.6	Yes
Boyra $\rightarrow$ Sonadanga	5.2	301	12.2	Yes
Khalishpur $\rightarrow$ Shibbari	6.7	389	15.4	Yes
Rupsha $\rightarrow$ Khalishpur	8.8	502	19.1	Yes
KUET Gate $\rightarrow$ Daulatpur	10.3	587	21.8	Yes
KUET Gate $\rightarrow$ Ferrighat	12.1	643	24.7	Yes
Average	**7.6**	**419**	**15.6**	**100%**

4.5.2 A* Algorithm Performance

Table 4.3 shows A* results for ten route pairs across the Khulna road network.

All queries succeeded. Average time: 15.6 ms. We were shooting for 100 ms, so the algorithm takes about $\frac{3}{5}$ of a second. Obviously, the longer the travelled distance, the more nodes we visit. But even the farthest query (Ferrighat, 12.1 km) took less than 25 ms. That's imperceptible on a mobile device.

4.5.3 AI Chatbot and Schedule Extraction

The average time it takes the chatbot to respond is 2.4 seconds. This includes the time spent calling the OpenAI API (2.1 s) — it only takes 0.3 s to query Firestore. To test the chatbot, we got 10 classmates to chat with the chatbot for a couple of minutes and ask bus-related questions. They got a response of 92% correct. This was because no schedule had been approved on that day, so the bot was guessing.

Extracting the schedule: See Table 4.4. GPT-4o has an accuracy of 94% on 20 emails. Route names were 100% every time. The hard ones were those in Bengali: it tried extracting those,

Table 4.4: Schedule Extraction Accuracy by Field (GPT-4o, 20 test emails)

Field	Correct	Accuracy
Bus Number	19/20	95.0%
Departure Time	18/20	90.0%
Route Name	20/20	100%
Direction	19/20	95.0%
Operating Days	18/20	90.0%
Overall	**94/100**	**94.0%**

Table 4.5: Objective Achievement Summary

Obj.	Statement	Evidence of Achievement
1	Hardware GPS+GSM tracking unit	TTGO T-Call deployed; SIM800L GSM data; 3.8 s mean update latency; $\pm$3.1 m GPS accuracy (Table 4.2)
2	Flutter mobile app with live map	App fully functional on Android; live bus marker updating on OSM tile layer
3	Automated schedule extraction	94% field-level accuracy on 20-email test set (Table 4.4); admin approval workflow implemented
4	A* boarding point recommendation	15.6 ms average query time; 100% path-found rate on 10 test pairs (Table 4.3)
5	AI chatbot with live context	GPT-4o integrated; 92% user satisfaction; 2.4 s mean response time
6	Admin dashboard	Full CRUD for buses, routes, schedules, notices, GPS feeds; role-based access enforced

but got the departure time incorrect in two emails, and days of operation incorrect in two emails.

4.6 Objective Achieved

Table 4.5 goes through each objective from Section 1.3 and shows what we built and what the evidence says.

4.7 Conclusion

Six objectives, six met. The GPS is fast enough to be useful in practice. A* runs in milliseconds on a phone. The chatbot is good at answering schedule questions as long as a schedule has been approved. The email pipeline turns what used to be a manual copy-paste

job into a two-tap approval. The numbers show it is not wishful thinking.

CHAPTER V

Societal, Health, Environment, Safety, Ethical, Legal and Cultural Issues

5.1 Intellectual Property Considerations

All of the work in this project, including the app, firmware, Firebase settings, and preprocessing was by us. We did not plagiarise. But we used open and commercial products as our starting point, so it's important to clarify who owns what.

Flutter uses BSD3 and OpenStreetMap uses ODbL attribution. We have done this in the About page of the app. OpenAI API is a commercial service (OpenAI Terms of Use) while Firebase is subject to Google's Terms of Service. Before finalising the packages in `pubspec.yaml` we checked the licences of the packages we installed and none were prohibited from academic or future commercial use.

With regards to ownership: the software and designs we developed are ours and KUET's, according to the university organisational policy for student projects. The design of TTGO-T-Call hardware is publicly shared by LilyGo and we are not guilty of IP theft in that case.

5.2 Ethical Considerations

Our biggest concern was: what and who are we tracking? We store a student's Google email and name, KUET ID, department and batch, phone number, blood group, hometown and optional profile photo when he or she signs up. This is more than is necessary, but it is all optional and private to the student and admins. We protect it via Firebase, which stores it encrypted at rest and in transmission; and we leave students to update and delete their data as they wish.

We made sure not to track the students. If a student taps "My Location", the app uses the coordinates they have on their phone to determine the A^* route - we don't receive anything. No one at KUET can even see where a student has been. We went through this issue, since the alternative - sending the student's coordinates to our server to improve suggestions about where to board - would have been more accurate but more complex.

The chatbot question is more complex. GPT-4o is good but not perfect, and if a student makes a decision based on an incorrect departure time, they miss their bus. We built the app to show a message on the chat interface ("AI assistant - check important details from official sources") and we built the admin approval process so that a human checks the schedule generated by AI before it is presented to the student. We could have done this automatically, but did not want to.

We do not track, data mine or advertise. The app does only one thing: help students get on the bus.

5.3 Safety Considerations

We once discussed a scenario: a student walking to the bus stop and looking at the live map and getting hit by traffic. This is not farfetched — it occurs with existing maps apps. We designed our live map so that once it loads, the bus marker automatically moves and the student does not need to tap or scroll. The phone does not tell the user every couple of seconds to check the screen. There is also a single line in the onboarding that warns about not using the phone when crossing the street. No doubt most students will ignore it, but it is there.

On hardware safety: the TTGO T-Call is in a 3D-printed box screwed into the bus interior, powered from a USB port. The enclosure prevents accidental shorts. The LiPo provides power if there is a temporary loss of USB power, which does occur when the bus engine is starting.

One last point: the GPS accuracy is ± 3.1 m in open sky. That is fine for a map dot. It is not sufficient to tell which lane the bus is in or to guide safety-critical decisions. We do not say it is, and our UI makes no such promises.

5.4 Legal Considerations

We did not wait for a Bangladeshi data protection law like GDPR to be passed, but implemented GDPR principles as a minimum: only collect what is required, use it only for the specified purpose, and allow users to delete their data on request. If Bangladesh passes a data protection law that covers apps like ours, the current design is already close to compliant.

The Android app uses three permissions: `ACCESS_FINE_LOCATION` and `ACCESS_COARSE_LOCATION` (only when the student seeks boarding-point guidance) and `READ_MEDIA_IMAGES` (only when the student uploads a profile picture). Both are requested with an explanation at runtime, not silently at install. They are declared in `AndroidManifest.xml` as required.

We run the Firebase project on the Blaze pay-as-you-go plan and use the OpenAI API within

its usage policies. The chatbot is never used to generate false information, and is clearly marked as AI in the app. We are not aware of any other legal issues this project raises.

5.5 Impact on Societal, Health, and Cultural Issues

Let me explain how unequal information on campus works: it is not much of a problem until you miss the bus because the schedule has changed. It is a pain for newenrollments, unsocialised students and those who are not connected to the right sphere of acquaintances. The app removes that asymmetry. Everyone gets the same information in the same way at the same time, as soon as an admin approves it.

There's a small health benefit too. It is not only a waste of time to stand in the summer sun of Khulna for 15 minutes waiting for the bus when you could have checked the app and left 15 minutes later; it's downright painful. The benefit is greater for those who do not have the opportunity to go to a comfortable room and wait until the last minute.

The chatbot is helpful for students who are new to the campus, and have not learned the social networks of the campus. Instead of finding out who can tell them how to catch the bus, they can simply ask the app. It's a small ask, but it's a big help.

5.6 Environmental and Sustainability Considerations

On average, the tracker uses around 200-250 mA. The SIM800L peaks with about 2 A for a brief period during a GPRS burst, but it is otherwise idle. The LiPo battery lasts for a whole day and the overall power consumption is insignificant relative to the bus engine.

At KUET they have printed schedule slips and notices. The app doesn't stop printing, but it eliminates it over time. That's one step towards reducing daily paper consumption.

The second, less visible result, is mode choice. If students know when (and where) the bus is, they are less likely to take a rickshaw or CNG auto-rickshaw if they are not sure. We have no exact measure of this impact, but it appears to be positive.

The TTGO T-Call is an off-the-shelf development board. It should stand up to a 3-5 years of normal use, and can be easily replaced. We did not use special or unrepresentative components.

CHAPTER VI

Solving Complex Engineering Problems and Conducting Activities

CHAPTER VII

Complex Engineering Problems with the Current Project

Some of the most difficult parts of this project didn't involve the components, but the interactions between the components, where two different fields meet and the answer is far from obvious.

A 5-Second Latency Budget, Four Systems and a Deadline

The problem, "send GPS coordinates to the map in 5 seconds or less" doesn't sound difficult, at first. But each step in the process takes some time: the GPS must take a "fix", the SIM800L modem must establish a GPRS connection, time is spent on TLS session startup (200-400 ms), Firestore data must make it through the Firestore CDN to the client, and then the Flutter UI must render the changed widget tree. None of this can be optimised on its own because we have a problem that crosses embedded firmware, telecommunications, cloud and mobile applications - four different fields. So we profiled each step separately to find what was taking the time, and then iterated back and forth getting overall execution time below the budget.

The biggest issue was the SIM800L connection time. The modem can take 2-3 seconds to reconnect to the GPRS bearer with a brief moment of signal loss, which alone can often exceed the budget. We ended up not closing the bearer, but reusing it, which hasn't been without complications - state management in the firmware and a watchdog timer to reset hangs.

Running A* on a Phone without Blocking the Map

The road network of Khulna has about 45,000 nodes. A real challenge is to run A* on this model on an ARM Cortex-A53 mid range Android phone ensuring the live map does not get laggy: it is not possible to run the search and update the map in the same thread as Flutter updating the screen. So we thought of using Dart's `Isolate` in order to run the search on an isolate (i.e. background thread). This is not difficult, but it turned out the data transfer cost between isolates is significant: simply serialising the entire graph on each search was insufficient.

We also had to implement a binary min-heap. The existing Dart sorted data structures don't support updates of priorities, which A* requires frequently, when it draws shorter paths. The

standard approach of inserting the item with a new lower priority and popping old items doesn't work, but would double the heap actions. We lower this to the expected $O(\log n)$ with our heap, which has a hash map index. We had to rewrite the code (three times) until the search was fast, memory use low and the UI snappy.

The Schedule Pipeline: Take the AI's Word or Improve its accuracy?

The email problem is one example of something that is ultimately a trust issue: what to automate and what to review? GPT-4o can convert unstructured information (text messages) into structured data, but it makes mistakes. We don't want just a wrong bus departure time: if a student is late to the bus, they can miss the exam or class. We could not simply automate and hope it would be accurate.

Approval by the administrator is an engineering solution: the AI is doing all the work the admin would have done in 10 minutes of copy-pasting, but it is not publishing anything until it is approved. It's a trade-off between efficiency and trust. 30 seconds to approve is a lot shorter than an hour of copy-pasting.

Security Rules in a Serverless Database

We do not have a server-side API that queries Firestore — the app accesses Firestore directly. That means all security logic has to be written in Firestore's declarative rules language, hosted on Google's servers. This is not as straightforward as it sounds: students should be able to read any approved schedule but not write one; admins should be able to perform cross-document transactions without the rules rejecting their writes mid-transaction; no user should be able to read another user's profile; and the unique-bus-number index collections need to be admin-only. It took careful experimentation in the Firebase Emulator Suite's rules simulator — which lets you test individual read and write scenarios against the rules without touching live data — to develop rules that satisfy all of these without unintentional loopholes.

7.1 Complex Engineering Activities for the Project

Working With Five Concurrently Built Systems

The hardware, Firebase backend, Flutter app, AI pipeline, and routing engine were all being developed simultaneously by the two of us — and they all had to communicate with each other. Each system had its own data model and failure modes. When the hardware side used `lat/lng` keys but the app expected `latitude/longitude`, nothing crashed — the marker just silently stopped updating. To find these gaps we agreed on shared Dart model classes

(`BusLocation`, `BusSchedule`, `BusRoute`, `Notice`, `Student`) from the start and did a lot of end-to-end integration testing. Most of the bugs we found were not inside a single component but in one component's assumptions about another.

Making the Tracker Last an Entire Day

The KUET bus runs between 8 - 10 hours a day. The reference tracker had to be up and running on LiPo for that long if it was not connected to an external power supply. The main cause was the SIM800L: when it bursts data across the GPRS network it can draw up to 2 A. That is sufficient to run a LiPo out of charge after an hour or so.

We calculated the power consumption first: 2 A for 100 ms every 3 seconds to transmit a packet, about 80 mA in idle mode (SIM800L) and finally 40 mA (ESP32). We then switched the ESP32 to light-sleep mode when not sending a message and added a sleep/wake routine on the SIM800L. This brought the total average power down to 100 - 150 mA. We tested this first with a USB bench power monitor and were pleased to then try it on the bus. A 3000 mAh LiPo at this average current can last for 20 hours, longer than any day long set of operations.

UNIQUE Keys in a Database Without Them

Firestore has no `UNIQUE` constraint. When two admins add a bus, with the same bus number, they will be accepted and the database will be invalid. SQL would reject one.

We have index collections (`unique_bus_numbers` and `unique_route_names`) to help with this, and Firestore transactions to keep everything in order. The add code reads the index to see if a route name is already taken, then either it proceeds or it signals a conflict - and all of this is inside of a transaction so that two adds can't both proceed with the same name. After a couple of tries and some experimentation the Firebase Emulator, we were satisfied that this worked for concurrent writes and deployed it.

Adapting the Map to A*

The original OpenStreetMap download for Khulna bounding box was not suitable. It consisted of multiple disjoint sub-graphs, a few-centimetre-long duplicate nodes (intersection modelled twice), self-loop edges and intersections where roads cross, but without an intersection node to define that they do; A* could not turn there. Any of these issues result in either crashes or incorrect routes.

To address this, we wrote a Python script to pre-process the graph: combine nodes closer than 2 m, remove edges from intersections (which split an edge in half), remove all nodes not in the largest connected component, and compute all the Haversine distances in advance.

This generates the file `khulna.json` that is part of the app. It takes the script 4 minutes to run on a laptop. It was arduous and painstaking, but the reason why A* finds the right route on the real road network.

CHAPTER VIII

Conclusions

8.1 Summary

We set out to build something to solve a problem for us, and instead ended up with a pretty extensive system. A GPS receiver, a real-time map, an AI chatbot and a schedule extractor, a path finder, all linked together, all written from scratch, all for the quick and dirty cost of a student project.

The results were promising: 3.8 s GPS delay, 15.6 ms path queries, 94% schedule extraction accuracy, 92% chain bot accuracy. Not perfect, but good enough to be useful for the intended use case of the development of the system.

What we did not anticipate was how easy it is to create something useful with not-so-expensive hardware and some existing APIs. With a circuit board and an OpenAI API key, we were able to build an app that would be tens of thousands of dollars to have such a system installed by an IT specialist. This is what we would like students to learn from our project.

8.2 Limitations

Things that aren't ready:

1. **One bus**: Only one bus is being tracked. The Firestore document structure allows multiple bus documents, but additional TTGO T-Call devices need to be acquired, programmed and set up to cater to the entire KUET fleet.
2. **2G GSM-only**: SIM800L module supports only 2G (GPRS) data. Which may be an issue in parts of the country where 2G is being switched off by telecommunication operators. A 4G module (e.g. SIM7600) would be better.
3. **GPS indoors**: The NEO-6M GPS receiver loses satellite fix in buildings or inside tunnels. Bus stop locations near buildings are more uncertain.
4. **Static road map**: The Khulna GeoJSON is from April 2024. Changes in the road network (new connections, closures) require a new export and deployment of the asset file.
5. **No offline mode**: An always-on internet is required. Offline cache for schedule will help.

8.3 Suggestions for the Future

If the project is to be continued - by anyone - these are our top priorities:

1. **Deploying the fleet**: Buy and install TTGO T-Call GPSs for all KUET buses. Include a fleet map showing the positions of all the buses in the fleet.
2. **4G/LTE upgrade**: Upgrade to a 4G LTE Cat-1 (LTE1) module (SIM7600) from the current 2G GPRS (SIM800L) module. This can reduce GPS position update time to less than 1 second, allow operation in regions where 2G is being phased out, and increase update frequency.
3. **Predictive ETA**: Incorporate a Kalman-filter-based predictor of the bus's direction and speed that estimates the bus's position ahead of time, enabling smooth tracking of the vehicle on the map and accurate prediction of the ETA.
4. **Road network change-detection**: Incorporate an OSM change-detection script to automatically update and deploy the Khulna GeoJSON file to accommodate major road-network changes.
5. **Student crowd-sourcing**: Allow students to report delays or malfunction of the bus service by using the app, then use this crowd-sourced data as a supplement to the GPS bus data.
6. **GTFS compliance**: Output KUET bus schedule and route data in Google's General Transit Feed Specification (GTFS) format to facilitate integration with other transit apps (Google Maps, Apple Maps) and transit research datasets.
7. **Live bus information on smartwatches**: Create a Wear OS smartwatch app to view bus arrival times without having to pull out the phone while waiting at bus stops.

CHAPTER A

Firebase Security Rules

These are the Firestore security rules that enforce the role-based access described in Chapter III. They are deployed through the Firebase CLI and run server-side on every database read and write.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    function isAuth()  { return request.auth != null; }
    function uid()     { return request.auth.uid; }
    function isAdmin() {
      return isAuth() &&
        get(/databases/$(database)/documents/users/$(uid()))
          .data.role == 'admin';
    }

    // Users - each user reads/writes only their own doc; admin reads all
    match /users/{userId} {
      allow read:  if isAuth() && (uid() == userId || isAdmin());
      allow write: if isAuth() && uid() == userId;
    }

    // Public read collections
    match /schedules/{doc}   { allow read: if isAuth(); allow write: if isAdmin(); }
    match /routes/{doc}      { allow read: if isAuth(); allow write: if isAdmin(); }
    match /buses/{doc}       { allow read: if isAuth(); allow write: if isAdmin(); }
    match /notices/{doc}     { allow read: if isAuth(); allow write: if isAdmin(); }
    match /live_schedules/{doc} { allow read: if isAuth(); allow write: if isAdmin(); }

    // Pending schedules - admin only
    match /pending_schedules/{doc} { allow read, write: if isAdmin(); }

    // Bus locations - admin writes; students read
    match /bus_locations/{doc} { allow read: if isAuth(); allow write: if isAdmin(); }

    // Unique index collections - admin only
    match /unique_bus_numbers/{doc}  { allow read, write: if isAdmin(); }
    match /unique_route_names/{doc}  { allow read, write: if isAdmin(); }
  }
}
```

CHAPTER B

Hardware Pin Configuration and Firmware Snippet

Table 2.1 lists the TTGO T-Call v1.1 pin assignments relevant to the GPS tracking firmware.

The core firmware loop (simplified) is shown below:

```
#define TINY_GSM_MODEM_SIM800
#include <TinyGsmClient.h>
#include <TinyGPSPlus.h>
#include <ArduinoHttpClient.h>

TinyGsm       modem(Serial1);      // SIM800L on UART1
TinyGsmClient gsmClient(modem);
TinyGPSPlus   gps;
HardwareSerial GPS_Serial(2);      // NEO-6M on UART2

void loop() {
  // Parse GPS NMEA sentences
  while (GPS_Serial.available())
    gps.encode(GPS_Serial.read());

  if (gps.location.isValid() && gps.hdop.value() < 200) {
    String payload = "{\"position\":{\"lat\":"
      + String(gps.location.lat(), 6)
      + ",\"lng\":" + String(gps.location.lng(), 6)
      + "},\"speed\":" + String(gps.speed.kmph(), 1)
      + ",\"ts\":\"" + getUTCString() + "\"}";

    // Send via SIM800L GPRS to Firebase REST API
    HttpClient http(gsmClient, FIREBASE_HOST, 443);
    http.patch(FIRESTORE_PATH, "application/json", payload);
    http.stop();
  }
  delay(3000);
}
```

Table 2.1: TTGO T-Call v1.3 Relevant Pin Configuration

Signal	**ESP32 GPIO**	**Description**
SIM800L TX	GPIO 27	SIM800L UART TX → ESP32 RX
SIM800L RX	GPIO 26	ESP32 TX → SIM800L UART RX
SIM800L RST	GPIO 5	SIM800L hardware reset (active low)
SIM800L PWR	GPIO 4	SIM800L power key
GPS TX	GPIO 12	NEO-6M UART TX → ESP32 RX2
GPS RX	GPIO 13	ESP32 TX2 → NEO-6M UART RX
LED	GPIO 2	Status LED (onboard, active low)
Battery	GPIO 35	Battery voltage divider (ADC)

REFERENCES

[1] M. Hassan *et al.*, “Gps/gprs-based bus tracking system for real-time vehicle monitoring,” *International Journal of Computer Applications*, vol. ???, no. ???, pp. ??–??, 2014.

[2] TransLoc, “Transloc university transit solutions,” https://transloc.com/, 2024, accessed: 2026-04-25.

[3] A. Khanna and R. Anand, “Iot-based smart bus tracking system using raspberry pi and mqtt,” *International Journal of Advanced Research in Computer Science and Software Engineering*, vol. 6, no. 6, pp. 1–6, 2016.

[4] A. Eltayeb *et al.*, “A firebase-based bus location tracking application for university transportation,” *Journal of King Saud University – Computer and Information Sciences*, vol. 33, no. ??, pp. ??–??, 2021.

[5] B. Ferris, K. Watkins, and A. Borning, “Onebusaway: A transit traveler information system,” *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pp. ??–??, 2010.

[6] A. Abutaleb *et al.*, “Using nfc events for bus location inference in campus transit systems,” *International Journal of Computer Networks and Communications*, vol. 7, no. 4, pp. ??–??, 2015.

[7] Y. Zheng *et al.*, “Llm-based public transit itinerary explanation with gtfs data,” arXiv preprint, 2024, cited conceptually in this report.

[8] H. Lin *et al.*, “Ai-assisted extraction of schedule information from pdf documents,” *Expert Systems with Applications*, vol. ??, no. ??, pp. ??–??, 2023.

[9] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[10] A. V. Goldberg and C. Harrelson, “Computing the shortest path: A* search meets graph theory,” *Proceedings of the 16th Annual ACM-SIAM Symposium on Discrete Algorithms*, pp. 156–165, 2005.

[11] H. Bast, D. Delling, A. Goldberg, M. M"uller-Hannemann, T. Pajor, P. Sanders, D. Wagner, and R. F. Werneck, “Route planning in transportation networks,” *In Algorithms and Combinatorics*, vol. 22, pp. 19–80, 2016.